



Rapport Technique Final — Projet ToolboxV8

Mastère Cybersécurité — Promotion 2025/2026



ToolboxV8

AYOUB BEN RACHED
TITOUAN AKA A MFOULA
ABDALLAH NASR

Table des matières

Table des matières

1. Présentation du projet.....	4
1.1 Contexte	4
1.2 Objectif général	4
1.3 Objectifs spécifiques (rappel cahier des charges)	4
1.4 Équipe	4
2. Analyse du besoin	4
2.1 Cartographie des parties prenantes	4
2.2 Menaces et surfaces d'attaque couvertes	5
2.3 Contraintes techniques.....	5
2.4 Matrice de risques	5
3. Organisation et méthodologie.....	6
3.1 Méthodologie en quatre grandes phases.....	6
3.2 Les quatre phases itératives	7
3.3 Répartition des tâches	7
4. Solution technique.....	8
4.1 Architecture globale	8
4.2 Stack technique retenue.....	8
4.3 Flux d'authentification	9
4.4 Image worker Kali multi-stage	9
4.5 Interface utilisateur	9
4.6 Pipelines CI/CD automatisés	10
4.7 Rapport automatisé (PDF / HTML / CSV)	11
4.8 Cloisonnement et segmentation réseau	11
5. Modules pentest et défensifs	12
5.1 Offensifs.....	12
5.1.bis Focus passive_recon (OSINT)	12
5.2 Défensifs	13
5.3 Module Response : pare-feu & remédiation	13
5.4 Upload de wordlists (spécifique Hydra / John).....	14
5.5 Implémentation des outils : principe et exemple	14
6. Tests, résultats et KPIs	14

6.1 Environnement de test	15
6.2 KPI temps d'exécution	15
6.3 KPI vulnérabilités détectées (mesures réelles end-to-end).....	15
6.4 KPI stabilité	16
6.5 KPI ergonomie.....	16
7. Sécurité et conformité	16
7.1 Authentification	16
7.2 Chiffrement.....	16
7.3 HTTPS avec Caddy (reverse proxy)	16
7.4 Audit et journalisation	18
7.5 Middlewares	18
7.6 Conformité RGPD et éthique	18
7.7 Politiques de sécurité	18
7.7.1 Politique de mots de passe	18
7.7.2 Politique d'authentification & session.....	18
7.7.3 Politique d'autorisation (RBAC)	18
7.7.4 Politique de journalisation (audit)	19
7.7.5 Politique de gestion des secrets	19
7.7.6 Politique d'utilisation éthique	19
7.7.7 Politique de mise à jour & dépendances	20
8. SIEM et supervision	20
8.1 Pipeline ELK.....	20
8.2 Règles Snort personnalisées (préparées)	20
8.3 Dashboard /siem.....	21
8.4 Pipeline Détection → Réponse (SOAR léger).....	21
8.5 Réponse à incident & documentation post-incident.....	22
8.5.1 Playbook de réponse à incident.....	22
8.5.2 Préservation des preuves (chaîne de traçabilité)	23
8.5.3 Modèle de rapport post-incident	23
8.5.4 Exemple appliqué : incident de brute-force SSH	24
9. Problèmes rencontrés et solutions (REX)	25
10. Conclusion et perspectives	26
10.1 Objectifs atteints	26
10.2 Limites actuelles	26

10.3 Perspectives d'évolution.....	27
11. Annexes	27
11.1 Correspondance livrables / cadre pédagogique	27
11.2 Correspondance livrables / cahier des charges	27
11.3 Arborescence du dépôt	28
11.4 Commandes utiles	28
11.5 Captures et artefacts fournis dans la remise ZIP	29
11.6 Annexe : extraits de code des intégrations outils.....	29
11.6.1 Reconnaissance (OSINT + active)	29
11.6.2 Scan de vulnérabilités	30
11.6.3 Exploitation.....	30
11.6.4 Web / API.....	31
11.6.5 Forensic (défensif, bonus)	32

1. Présentation du projet

1.1 Contexte

Le client est une société spécialisée en cybersécurité offensive. Elle réalise régulièrement des tests d'intrusion pour des entreprises privées et des institutions publiques. Aujourd'hui, ces tests reposent majoritairement sur des manipulations manuelles, ce qui les rend **longs** et **hétérogènes** selon les intervenants.

1.2 Objectif général

Développer **ToolboxV8**, une toolbox automatisée permettant de réaliser l'ensemble des étapes d'un pentest (reconnaissance, scan de vulnérabilités, exploitation, analyse web) avec une **interface simple**, des **modules réutilisables** et un **reporting standardisé**.

1.3 Objectifs spécifiques (rappel cahier des charges)

#	Objectif	Réponse apportée
1	Réduire ≥ 40 % le temps d'un pentest	Profils prédéfinis (chips) + lancement en 3 clics + reporting automatique
2	Standardiser les pratiques internes	Même flux pour tous les outils, format de sortie unique, rapport PDF type
3	Interface utilisable par analystes non-développeurs	Plus aucun textarea éditable : chips, dropdowns, upload de fichiers
4	Rapports lisibles et exploitables	Export PDF / HTML / CSV (PDF ReportLab par défaut, charte pro) avec sortie CLI brute par outil
5	Intégration simple dans l'écosystème	Stack 100 % Docker Compose, API REST Swagger documentée

1.4 Équipe

Membre	Rôle	Responsabilités
Ayoub Ben Rached	Architecte / Back-end / Sécurité	Architecture split api/web, orchestration Celery, sécurisation, défensif/SIEM
Abdallah NASR	Analyste / Intégration offensive & QA	Modules recon/scan/exploit, tests sur cibles Metasploitable, validation Kali
Titouan AKA A MFOULA	Interface & Reporting	Frontend Jinja2, UI chips + modale, générateur de rapports, UX

2. Analyse du besoin

2.1 Cartographie des parties prenantes

L'analyse du besoin part d'un constat : le client n'est pas un bloc homogène mais une organisation à **cinq pôles**, chacun exposé à des surfaces d'attaque différentes : applicative pour le SaaS, réseau pour l'infrastructure, communication pour le support, postes internes pour les RH. Une toolbox générique passerait à côté de ces spécificités. ToolboxV8 a donc été conçue pour que **chaque pôle retrouve les outils pertinents à son périmètre**, sans configuration manuelle. Le tableau ci-dessous établit cette correspondance besoin → outils mobilisés :

Pôle	Besoins	Outils ToolboxV8 mobilisés
Sécurité (SOC/EDR/XDR)	Tests ciblés, détection de flux	Nmap, Metasploit, ZAP, Snort (défensif)
Développement SaaS	Audit applications web et API	SQLmap, OWASP ZAP Spider + Active, Dependency-Check
Infrastructure	Réseaux et systèmes internes	Nmap NSE, Hydra, SSLyze, OpenVAS-like via

		--script=vuln
Support client	Sécurité des outils de communication	Nikto, SSlyze
RH / Administration	Pentest d'outils internes	John the Ripper (jumbo, 304 formats)

2.2 Menaces et surfaces d'attaque couvertes

- **Renseignement passif (OSINT)** : fuites publiques, profils sociaux, documents indexés, mentions de credentials → couvert par **passive_recon** (Google Dorks, 3 catégories : Mot-clé, Réseaux sociaux, Domaine).
- **Enumération et reconnaissance active** : exposition réseau, DNS, ports ouverts, versions vulnérables → couverte par **recon**.
- **Vulnérabilités applicatives** : CVE connues, configurations faibles → **scan** (Nmap NSE + Nikto) et **web_scan** (ZAP).
- **Authentification** : mots de passe faibles, absence de MFA → **exploit** mode Hydra (online) + mode John (offline sur hashes fuités).
- **TLS/SSL** : ciphers obsolètes, certificats expirés, CVE TLS → **scan** mode SSlyze (Cert/Standard/Full).
- **Injection SQL, XSS, CSRF, SSRF** → **exploit** mode SQLmap + **web_scan** Active.
- **Exploitation post-scan** : validation de la criticité → **exploit** mode Metasploit (handler, EternalBlue, portscan, smb_ms17_010).

2.3 Contraintes techniques

- **Conformité RGPD / éthique** : toute action sensible journalisée (**AuditLog**), pas de collecte de données clients, avertissements explicites dans l'UI.
- **Isolation des outils offensifs** : tournent dans un conteneur worker Kali isolé, sans accès direct aux bases métiers.
- **Reproductibilité** : stack **docker-compose** déployable en une commande (**./scripts/start.sh** ou **.\scripts\start.ps1**).

2.4 Matrice de risques

Évaluation des menaces pesant sur le SI cible (et sur la toolbox elle-même), cotées selon **Vraisemblance** × **Impact** sur une échelle 1 (faible) à 5 (critique). Le **niveau de risque** = Vraisemblance × Impact (vert ≤ 6, orange 8-12, rouge ≥ 15). La dernière colonne indique le traitement apporté par ToolboxV8.

#	Menace	Vraisemblance	Impact	Risque	Traitement / Couverture ToolboxV8
R1	Injection SQL (SQLi) sur applications web/API	4	5	(R) 20	Détection : exploit SQLmap + web_scan ZAP Active ; règle Snort UNION SELECT (sid 9000004)
R2	RCE / exploitation de CVE (ex. Log4Shell)	3	5	(R) 15	Détection : scan Nmap NSE --script=vuln ; règle Snort \$_jndi (sid 9000009) ; validation via exploit Metasploit
R3	Brute-force / mots de passe faibles (SSH, HTTP, services)	4	4	(R) 16	Détection : exploit Hydra (online) + John (offline) ; règles Snort brute-force SSH/HTTP (sid 9000002/3) ; politique MDP bcrypt (\$7.7.1)
R4	XSS / CSRF / SSRF applicatifs	4	3	(O) 12	Détection : web_scan ZAP Active ; règle Snort <script> (sid 9000005)
R5	Phishing / ingénierie sociale (vol de credentials)	4	4	(R) 16	Renseignement amont : passive_recon (détection de fuites de credentials, profils

					exposés, dorks leaked/dump ; MFA recommandé (perspective §10.3 SSO/OIDC)
R6	DDoS / déni de service sur les services exposés	3	4	(O) 12	Atténuation : reverse proxy Caddy (rate-limiting activable), ResponseModule.block_ip (iptables DROP) sur IP source ; supervision SIEM des pics de trafic
R7	TLS/SSL faible (ciphers obsolètes, cert expiré)	3	3	(O) 9	Détection : scan SSLyze (Cert/Standard/Full) avec pré-check TCP IPv4/IPv6
R8	Exposition réseau / ports & services non maîtrisés	4	3	(O) 12	Détection : recon Nmap + scan ; règle Snort scan SYN (sid 9000001)
R9	Fuite d'informations (OSINT) : documents indexés, credentials publics	4	3	(O) 12	passive_recon : 24 dorks (Mot-clé / Réseaux sociaux / Domaine) pour cartographier l'exposition publique avant correction
R10	Compromission de la toolbox elle-même (vol de token, accès non autorisé)	2	5	(O) 10	JWT HS256 + cookie HttpOnly + RBAC 3 rôles + Fernet + audit logs (§7) ; HTTPS Caddy ; split api/web
R11	Fuite de secrets (clés, MDP en clair dans le repo)	2	4	(V) 8	.gitignore strict (.env , *.key , *.pem), secrets en variables d'env, jamais loggés (§7.7.5)
R12	Usage offensif non autorisé / non-conformité RGPD	2	4	(V) 8	Bandeau d'avertissement UI + PDF, traçabilité audit_logs , profils "Quick" par défaut (§7.7.6)

Lecture : les risques rouges (R1, R2, R3, R5) concentrent l'effort de détection ; ToolboxV8 couvre **les 12 menaces** soit par un module offensif (identification de la vulnérabilité), soit par une brique défensive (détection Snort / réponse / politique de sécurité). Les menaces non détectables techniquement (phishing) sont traitées en amont par le renseignement passif et en aval par les recommandations.

3. Organisation et méthodologie

3.1 Méthodologie en quatre grandes phases

Plutôt qu'un Scrum rigide en sprints chronométrés (peu réaliste pour une équipe de trois menant le projet en parallèle de ses cours), l'organisation a reposé sur **quatre phases itératives** étalées sur **six mois**, **chacune livrant un incrément fonctionnel** de la toolbox (approche incrémentale), avec une coordination **au fil de l'eau** :

- **Communication** : échanges à l'oral sur **Microsoft Teams et Discord**, selon le besoin, sans cérémonies imposées.
- **Versioning** : code sur **GitHub avec un miroir GitLab** : une branche **main** stable et protégée, le travail en cours isolé sur des branches dédiées.
- **Qualité** : chaque outil est **testé sur une cible réelle mais autorisée** avant son intégration dans **main** : VM Metasploitable2 et conteneurs vulnérables que nous contrôlons, ainsi que des plateformes de test publiques explicitement prévues pour le pentest (sites Acunetix ***.vulnweb.com**, **scanme.nmap.org**, **badssl.com**). Aucune action n'a été menée sur un système tiers sans autorisation.

3.2 Les quatre phases itératives

La progression a suivi une logique de **risque décroissant** : sécuriser d'abord le socle technique, puis empiler les capacités offensives et défensives, et enfin fiabiliser la restitution et les démonstrations. Chaque phase s'est conclue par un **incrément démontrable** (une capacité de la toolbox réellement utilisable), dans l'esprit d'une livraison incrémentale.

Phase	Périmètre	Livrables clés
1. Socle	Fondation technique	Architecture Docker, squelette FastAPI, authentification JWT, base PostgreSQL, orchestration Celery
2. Offensif	Les 5 modules d'attaque	passive_recon, recon, scan, exploit, web_scan (Nmap, Nikto, SSLyze, SQLmap, Hydra, John, Metasploit, ZAP, Gobuster), sur image worker Kali Rolling
3. Défensif	Détection & réponse	SIEM ELK (indexation des scans), règles Snort, module response (block_ip / iptables), dashboard /siem
4. Reporting & finitions	Restitution & durcissement	Générateur de rapports PDF/HTML/CSV, HTTPS via Caddy, CI/CD (GitHub Actions + GitLab CI), cibles vulnérables intégrées, démos end-to-end

Plusieurs briques ont été **retravaillées en cours de route** à la suite des retours d'expérience : refonte de l'authentification en **cookie HttpOnly + split api/web**, ajout du module **OSINT passive_recon**, refonte du scan **ZAP en polling** (au lieu du « fire-and-forget »), et mise en place de **Caddy** et de la **CI/CD**. Le détail de ces incidents et de leurs solutions figure au **§9 (REX)**.

3.3 Répartition des tâches

Si chaque membre porte un axe principal (cf. §1.4), la répartition réelle a suivi un principe d'**appropriation partagée** : tous ont touché au développement, aux tests et à la documentation, afin de garantir la cohérence du produit et la capacité de chacun à en présenter l'ensemble en soutenance. Concrètement :

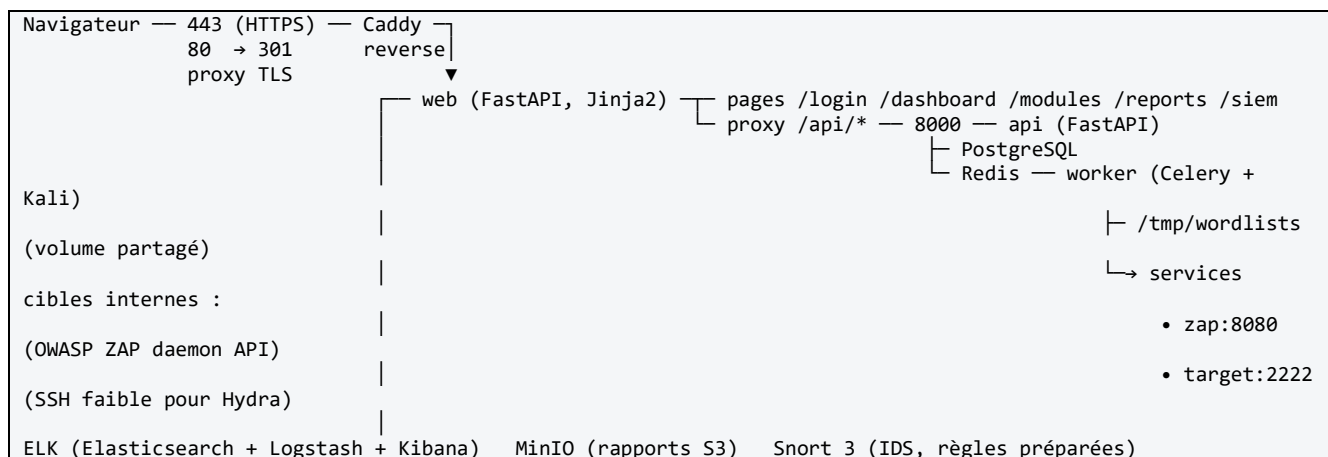
- **Ayoub Ben Rached** a piloté l'architecture back-end, l'orchestration Celery et la sécurisation (authentification, HTTPS, SIEM/défensif).
- **Abdallah NASR** a mené l'intégration des outils offensifs et la qualité (tests sur cibles Metasploitable, validation des profils et des démos).
- **Titouan AKA A MFOULA** a pris en charge l'interface Jinja2, l'ergonomie analyste (chips, modales) et le générateur de rapports.

La coordination se faisait au fil de l'eau, à l'oral, sur Teams et Discord (cf. §3.1).

4. Solution technique

4.1 Architecture globale

Trois services applicatifs **FastAPI/Celery** + une stack de supports, tous **conteneurisés** (Docker Compose) :



Justification du split `api` / `web` :

- Isolation : l'API reste accessible pour intégrations externes sans exposer l'UI.
- Auth sécurisée : le cookie HttpOnly n'existe que sur le service **web** (pas de token JS accessible, protection XSS).
- Surface d'attaque réduite : l'API accepte seulement **Authorization: Bearer**. Un client qui tombe dessus ne peut pas y accéder sans token.
- Évolutivité : on peut scaler indépendamment le rendu HTML et les appels métier.

4.2 Stack technique retenue

Couche	Technologie	Choix justifié
Langage	Python 3.11	Demandé par le cahier des charges, standard offensif
Framework web	FastAPI	Async, typage Pydantic, Swagger auto, perf
Templating	Jinja2	Léger, HTML idiomatique, compatible WeasyPrint / ReportLab
ORM	SQLAlchemy 2.0	Requêtes typées, migrations via Alembic possibles
Tâches	Celery 5 + Redis 7	Orchestration asynchrone des scans longs
DB	PostgreSQL 16	Relationnel robuste, JSONB pour ScanJob.result
Stockage	MinIO (S3)	Découpler les rapports du FS local
SIEM	Elasticsearch 8.13 + Logstash + Kibana	Stack standard, ingestion Snort directe
IDS	Snort 3	Signatures ouvertes, règles locales personnalisables
Worker	Kali Rolling (Docker officiel)	Tous les outils offensifs pré-packagés par Offensive Security
Reporting	ReportLab 4	Mise en page professionnelle, preformatted pour CLI brut
Reverse proxy TLS	Caddy 2 (caddy:2-alpine)	HTTPS auto via CA interne, config 3 lignes, basculement Let's Encrypt en 1 ligne pour la prod
CI/CD	GitHub Actions + GitLab CI (parallèle)	Lint → tests pytest avec PG/Redis → build Docker, à chaque push
Scanner web	OWASP ZAP 2.17 daemon (zaproxy/zap-	API REST sur :8080, intégré au compose,

	stable)	polling + récupération auto des alertes
Cible vulnérable	linuxserver/openssh-server (pentest_target)	SSH faible pentest_user:toor sur :2222, démos Hydra réutilisables
Containers	Docker Compose v2	Lancement en une commande

4.3 Flux d'authentification

1. GET /login	→ web sert login.html
2. POST /login (form user+pass)	→ web → api POST /api/auth/token
3. api valide credentials	→ {access_token: <JWT>}
4. web pose cookie HttpOnly	→ Set-Cookie: access_token=<JWT>; HttpOnly; SameSite=Lax; Max-Age=3600
5. 303 See Other → /dashboard	→ cookie envoyé automatiquement
6. dashboard → JS fait fetch('/api/...')	
7. web proxy lit cookie	→ injecte Authorization: Bearer → api authentifié

Les **clients externes** (CI, scripts offsec) continuent d'utiliser **POST /api/auth/token** directement sur le port 8000 avec un Bearer classique, sans aucune régression.

4.4 Image worker Kali multi-stage

Build en deux étapes pour contourner PEP 668 de Kali (environnements externes verrouillés) :

Stage 1 (python:3.11-slim + poetry) : export requirements.txt
Stage 2 (kalilinux/kali-rolling) : apt outils + pip install -r requirements.txt dans /opt/venv

Outils pré-installés : **nmap 7.99**, **nikto 2.6**, **sqlmap 1.10**, **hydra 9.6**, **john 1.9 jumbo** (304 formats), **sslyze**, **msfconsole 6.4**, **whois**, **dig**, **rockyou.txt** (134 Mo décompressé).

Services clés du `docker-compose.yml` ([docker/docker-compose.yml](#)) : le worker porte les outils ; ZAP et la cible vulnérable y sont intégrés pour les démos :

worker:	# Celery + Kali : tous les outils offensifs
build:	
dockerfile: docker/Dockerfile.celery	
volumes:	
- wordlists_data:/tmp/wordlists	# wordlists uploadées, partagées avec l'api
networks: [pentest_net]	
zap:	# OWASP ZAP en démon API (module web_scan)
image: zaproxy/zap-stable	
command: >	
zap.sh -daemon -host 0.0.0.0 -port 8080 -config api.key=zapsecret	
networks: [pentest_net]	
target:	# cible vulnérable SSH (demos Hydra ; John = hash dédié)
image: linuxserver/openssh-server:latest	
environment:	
USER_NAME: "pentest_user"	
USER_PASSWORD: "toor"	# volontairement faible, isolé du LAN
networks: [pentest_net]	

4.5 Interface utilisateur

Principe : chaque module est un formulaire minimal (cible, port si besoin, profil), les chips remplacent les textareas. Plus aucune variable **{target}** à substituer mentalement.

Pages (Jinja2) :

- **/login** : formulaire simple, gestion d'erreur server-side

- **/dashboard** : KPIs + jobs récents + rapports récents + raccourcis vers les 5 modules
- **/modules** : 5 cartes (**passive_recon**, recon, scan, exploit, web_scan) avec config dynamique par outil ; passive_recon mis en première position car c'est l'étape la plus en amont d'un pentest (OSINT avant toute interaction réseau)
- **/reports** : liste + génération (PDF par défaut depuis l'UI ; export HTML et CSV également disponibles via l'API)
- **/siem** : graphiques Chart.js sur Elasticsearch

Composants clés du frontend :

- **Chips de profils** (**profileBlock** JS) pour Nikto, SSLyze, SQLmap, MSF, ZAP, Nmap NSE
- **Catalogue de dorks** (passive_recon) : 24 templates répartis en 3 catégories (6 Mot-clé + 6 Réseaux sociaux + 12 Domaine) avec sélection multiple par checkbox + zone de dorks personnalisés **{target}**
- **Modale wordlist** pour Hydra + John : 3 boutons (fichier / manuelle / rockyou.txt)
- **Polling Celery** (3 s) pour la progression temps réel avec barre et logs
- **Validation de cible adaptative** : validation stricte IP/domaine/URL pour les modules réseau, désactivée pour passive_recon (champ libre type "nom de personne", "marque", "sujet") et pour John the Ripper (hash)

Polling temps réel ([frontend/static/js/main.js](#)) : il interroge l'API toutes les 3 s jusqu'à la fin du job Celery, met à jour la barre de progression et les logs, puis s'arrête :

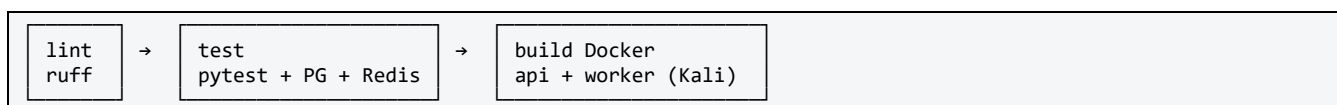
```
function pollJob(jobId, onUpdate, intervalMs = 3000) {
  const id = setInterval(async () => {
    const data = await apiFetch(`/modules/jobs/${jobId}`);
    onUpdate(data); // maj barre + logs live
    if (['done', 'error'].includes(data.job?.status)) {
      clearInterval(id); // stop quand terminé
    }
  }, intervalMs);
  return id;
}
```

4.6 Pipelines CI/CD automatisés

Deux pipelines en parallèle, déclenchés à chaque **git push** et chaque pull/merge request :

Plateforme	Fichier	Statut
GitHub Actions	.github/workflows/ci.yml	Actif (repo principal sur GitHub)
GitLab CI	.gitlab-ci.yml	Présent (compatibilité, push miroir GitLab via scripts/push_gitlab.sh)

Les deux pipelines exécutent **les mêmes trois étapes** :



Stage 1 : Lint (ruff) : analyse statique du code Python, en mode **--output-format=github** pour annoter directement les pull requests sur GitHub.

Stage 2 : Tests : pytest avec services Postgres 16 et Redis 7 en parallèle (équivalent à la stack **docker-compose** minimale). Variables d'environnement injectées :

```
env:
  DATABASE_URL: postgresql://pentest:pentest@localhost:5432/pentestdb
  REDIS_URL: redis://localhost:6379/0
  SECRET_KEY: ci-test-secret-key-not-used-in-prod
  FERNET_KEY: M0F5T3JuZXRLZX1Gb3JDSVRLc3RPbm5MzJiPT0=
```

Stage 3 : Build Docker : build des deux images (`docker/Dockerfile` pour api/web, `docker/Dockerfile.celery` pour le worker Kali) avec cache GitHub Actions (`type=gha`) pour accélérer les builds suivants. Pas de push sur un registry pour l'instant, ce point étant prévu en perspective (cf. §10.3).

Justification GitHub Actions + GitLab CI : le cahier des charges mentionne explicitement « CI/CD (GitLab) ». Le repo principal étant hébergé sur GitHub, GitHub Actions est le pipeline actif. Le `.gitlab-ci.yml` est conservé pour rester conforme à la lettre du cahier et permettre un futur mirroring sur GitLab sans réécriture.

4.7 Rapport automatisé (PDF / HTML / CSV)

Le générateur `app/reporting/generator.py` sait produire le rapport en **trois formats** : `pdf`, `html` et `csv` (paramètre `format` de `POST /api/reports/generate`, PDF par défaut ; le bon type MIME est servi au téléchargement). Le format **PDF (ReportLab)** est le plus abouti, structuré en 4 pages type :

1. Couverture : eyebrow `TOOLBOXV8 • RAPPORT AUTOMATISÉ`, titre, métadonnées
2. Encart CODIR orange (synthèse exécutive)
3. Statistiques + **déroulé technique par outil** (commande / sortie console / résultats / stderr / détails)
4. Tableau synthétique + recommandations + annexes + footer paginé

La fonction `_build_tools_sections(result)` normalise les champs `command`, `output`, `stderr`, `credentials/cracked`, `extras_json` pour chaque outil → la sortie CLI apparaît **telle qu'en terminal**.

4.8 Cloisonnement et segmentation réseau

Même déployée sur une seule machine en développement, ToolboxV8 applique un **cloisonnement réseau** strict, et son architecture est pensée pour une **segmentation en zones** en production, un point d'autant plus sensible que l'outil héberge un worker offensif et une cible volontairement vulnérable.

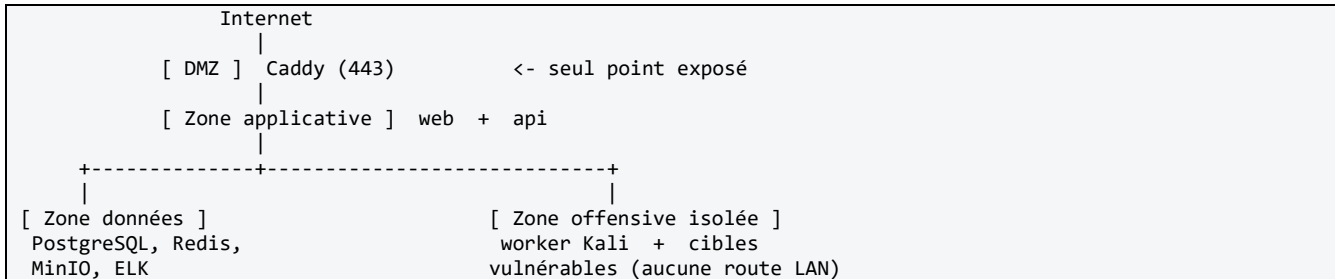
En l'état (développement), tous les conteneurs communiquent sur un réseau Docker **bridge privé** (`pentest_net`), isolé du LAN de l'hôte. Seuls les ports strictement utiles sont **publiés** ; les briques sensibles restent **internes** au réseau Docker :

Exposé sur l'hôte	Interne au réseau Docker uniquement
Caddy (80/443), api (8000), web (3000)	PostgreSQL, Redis, worker Kali
MinIO (9000/9001), Elasticsearch (9200), Kibana (5601) (debug)	ZAP, cible vulnérable <code>pentest_target`</code> (jamais exposée au LAN)

Deux garanties de sécurité en découlent :

- la **base de données**, la **file Redis**, le **worker offensif** et la **cible volontairement vulnérable** ne sont **jamais joignables depuis l'extérieur**, uniquement via le réseau interne ;
- le **worker Kali** (qui exécute les outils d'attaque) et la cible vulnérable sont confinés ensemble : une compromission de la cible de démonstration **ne déborde pas** sur le poste hôte ni sur le LAN.

Segmentation cible (production) : l'architecture se transpose en **quatre zones** cloisonnées (réseaux Docker distincts ou VLAN), derrière un point d'entrée unique :



En production, seul Caddy resterait publié ; les ports api/web/Elasticsearch/Kibana/MinIO ouverts en développement pour le debug seraient **fermés** (accès via le réseau interne ou un bastion). La **zone offensive** serait **strictement isolée** (pas de route vers la zone données métier), conformément au principe de **moindre exposition**. Le module `ResponseModule` (`block_ip` via iptables, §5.3) agit alors comme **pare-feu applicatif** au sein de ce cloisonnement.

5. Modules pentest et défensifs

5.1 Offensifs

Module	Outils	Profils UI
passive_recon	Google / Bing / DuckDuckGo dorks	Catalogue 24 dorks (Mot-clé : PDF, Office, intitle, CV, leaks, credentials ; Réseaux sociaux : LinkedIn, GitHub, Twitter, Reddit, Facebook, Pastebin ; Domaine : pages indexées, sous-domaines, PDF, Office, configs, SQL/logs, admin, login, directory listing, secrets, emails, errors) + dorks personnalisés
recon	Nmap, DNS, whois, WhatWeb	Nmap : Quick/Standard/Full TCP/Stealth
scan	Nmap <code>--script=vuln</code> , Nikto, SSLyze	Nmap NSE : Quick/Standard/Full/Safe ; Nikto : Quick/Standard/Full/Evasion ; SSLyze : Cert/Standard/Full (avec pré-check TCP IPv4/IPv6 + détection cible sans HTTPS)
exploit	SQLmap, Hydra, John jumbo, Metasploit	SQLmap : Quick/Standard/Aggressive/Dump ; MSF : Handler/EternalBlue/PortScan/SMB
web_scan	OWASP ZAP (Spider + Active, daemon API intégré au compose), Gobuster, Dependency-Check	ZAP Spider : Quick/Standard/Deep avec récupération des URLs découvertes ; ZAP Active : Quick/OWASP/Full avec polling jusqu'à 100% + récupération des alertes agrégées par sévérité ; Gobuster : Quick/Standard/Full sur wordlists dirb
post_exploit	(documentaire)	/

Détails des profils → [docs/modules.md](#).

5.1.bis Focus passive_recon (OSINT)

Le module `passive_recon` (`backend/app/modules/offensive/passive_recon.py`) génère des **Google Dorks** prêts à **exécuter** sans aucun trafic vers la cible (recherche **100 % passive**). Trois catégories :

Catégorie	Cas d'usage	Exemples de dorks
Mot-clé	Cible libre (nom, marque, sujet)	<code>"{target}" filetype:pdf, intitle:"{target}", "{target}"</code>

		("leaked" "leak" "dump")
Réseaux sociaux	Profils & présence	"{target}" site:linkedin.com, site:github.com, site:reddit.com, site:pastebin.com
Domaine	Cible = domaine ou IP	site:{target}, site:*. {target} - www, site:{target} (inurl:admin inurl:wp-admin), intext:"@{target}"

Le frontend permet de **cocher / décocher** chaque dork puis d'ouvrir tous les résultats en multi-onglets (un par dork). Le rapport PDF liste les requêtes générées et les URLs cliquables.

5.2 Défensifs

- **SIEM ELK** *(déployé)* : Elasticsearch + Logstash + Kibana opérationnels, indexation automatique des résultats de scans dans `pentest-logs-*`. Pipeline Logstash [siem/elk/logstash.conf](#) prêt à parser des alertes Snort (input `file` sur `/var/log/snort/alert` + filtre grok dédié).
- **IDS Snort 3** *(règles préparées, conteneur non déployé)* : 9 règles personnalisées écrites dans [siem/snort/local.rules](#) : scan Nmap SYN, brute-force SSH (5/60s), brute-force HTTP Basic, SQLi (**UNION SELECT**), XSS, Directory Traversal, signature Nikto, signature SQLmap, Log4Shell (CVE-2021-44228). Le conteneur Snort lui-même n'est **pas** dans `docker-compose.yml` actuel. Le déploiement a été repoussé pour contourner les contraintes de mode promiscuous sous Docker Desktop Windows/WSL2 (cf. §10.2).
- **Response active** *(module fonctionnel, déclenchement manuel)* ([backend/app/modules/defensive/response.py](#)) ; voir §5.3 et §8.4
- **Forensic** (bonus) : ClamAV + VirusTotal API v3

5.3 Module Response : pare-feu & remédiation

Le module `ResponseModule` expose 4 actions de remédiation, journalisées systématiquement dans le SIEM :

Action	Méthode	Implémentation
Blocage IP	<code>block_ip(ip, reason)</code>	<code>iptables -A INPUT -s <ip> -j DROP</code> dans le conteneur worker Kali
Déblocage IP	<code>unblock_ip(ip)</code>	<code>iptables -D INPUT -s <ip> -j DROP</code>
Isolation hôte	<code>isolate_host(ip)</code>	Hook prévu pour intégration hyperviseur (vSphere/Proxmox/AWS Security Group)
Alerte SIEM	<code>send_alert(message, severity)</code>	Indexation immédiate dans Elasticsearch (<code>response_action / alert</code>)

Mode dégradé : si `iptables` n'est pas disponible (cas d'un dev local Windows pur sans namespace réseau privilégié), `block_ip` retourne `{"status": "simulated"}` au lieu de planter. L'action est tout de même journalisée dans le SIEM pour traçabilité. Le worker Kali, lui, dispose de `iptables` nativement.

Journalisation systématique : chaque appel `block_ip/isolate_host/send_alert` est indexé dans Elasticsearch avant l'action elle-même → traçabilité même si la commande système échoue.

État actuel : le module est fonctionnel et invocable depuis le worker Celery ou un script admin. L'exposition API REST (`POST /api/defensive/response/block-ip`) est en backlog pour automatiser la chaîne complète (cf. §10.3).

5.4 Upload de wordlists (spécifique Hydra / John)

Nouvel endpoint `POST /api/modules/wordlist` (multipart). Fichier stocké dans un volume Docker partagé `/tmp/wordlists` accessible aux deux services `api` et `worker`. Utilisable comme `user_file`, `pass_file` ou `wordlist_file` dans le lancement du scan.

5.5 Implémentation des outils : principe et exemple

Plutôt que de piloter chaque outil par des options brutes, ToolboxV8 applique un **pattern d'intégration unique** à ses treize outils offensifs et défensifs. Le principe est le suivant : un dictionnaire de **profils** (matérialisés par les **chips** `Quick` / `Standard` / `Full` de l'interface) traduit le choix de l'analyste en **arguments de ligne de commande réels** ; l'outil est ensuite exécuté via `subprocess.run()` (ou son API REST pour ZAP et Metasploit) ; enfin la sortie est capturée **elle qu'elle apparaît en terminal**. Ce parti pris répond directement à deux exigences du cahier des charges : l'analyste n'a **aucune syntaxe CLI à mémoriser** (interface accessible aux non-développeurs), et le rapport restitue une trace d'exécution **fidèle et auditable**.

L'exemple du scan de vulnérabilités Nmap illustre le mécanisme : la **chip** choisie sélectionne une catégorie de scripts NSE, le reste de la commande étant assemblé programmatiquement :

```
_NMAP_VULN_PROFILES = {
    "quick":  ["--script=default"],
    "standard": ["--script=vuln"],
    "full":   ["--script=vuln,exploit,auth"],
    "safe":   ["--script=safe"],
}

def _nmap_vuln(self, target, options):
    profile = options.get("nmap_vuln_profile", "standard")
    script_args = self._NMAP_VULN_PROFILES.get(profile, ...)
    cmd = ["nmap"] + script_args + ["-Pn"] + port_arg + [target]
    proc = subprocess.run(cmd, capture_output=True, text=True, timeout=1200)
    return {"command": " ".join(cmd), "output": _strip_nmap_fingerprints(proc.stdout), ...}
```

Au-delà de la simple construction de commande, plusieurs modules réalisent une **analyse de la sortie** pour en extraire l'information à valeur ajoutée. Hydra, par exemple, parse les lignes de résultat afin de remonter directement les identifiants compromis dans le rapport, sans intervention humaine :

```
cmd = ["hydra", "-L", user_path, "-P", pass_path, "-t", threads] + [target, service]
proc = subprocess.run(cmd, capture_output=True, text=True, timeout=timeout)
creds = [l.strip() for l in proc.stdout.splitlines()]
        if "login:" in l and "password:" in l] # ex : login: pentest_user password: toor
```

***Catalogue complet** : pour ne pas alourdir cette section, les extraits d'intégration des **treize outils** (Nmap recon/NSE, Nikto, SSLyze, SQLmap, Hydra, John, Metasploit, ZAP, Gobuster, Dependency-Check, passive_recon, ClamAV/VirusTotal), regroupés par phase de pentest, sont fournis en **[Annexe §11.6](#116-annexe--extraits-de-code-des-integrations-outils)**.*

6. Tests, résultats et KPIs

La validation de ToolboxV8 ne s'est pas limitée à des tests unitaires : chaque module a été éprouvé **de bout en bout**, sur des cibles réelles et autorisées, jusqu'à la production d'un rapport. Cette section objective les résultats à travers **quatre familles de KPIs** (temps d'exécution, vulnérabilités détectées, stabilité et ergonomie) afin de mesurer concrètement les gains face à un pentest manuel et de vérifier l'atteinte des objectifs du cahier des charges (en particulier la réduction d'au moins 40 % du temps de réalisation).

6.1 Environnement de test

- **Hôte** : Windows 11 + Docker Desktop (WSL2)
- **Cibles publiques autorisées** (sites Acunetix volontairement vulnérables) :
 - testasp.vulnweb.com (ASP / IIS / MSSQL) : SQLmap, ZAP, Gobuster
 - scanme.nmap.org : Nmap NSE, Nikto, recon
 - badssl.com : SSLyze (audit TLS complet)
- **Environnements de test créés spécifiquement** pour démontrer les deux modules de cassage de mots de passe de façon **reproductible** à chaque démarrage de la stack (et donc à la soutenance) :
 - **pour Hydra** (brute-force *en ligne*) : un **conteneur SSH volontairement faible** intégré au **docker-compose** : **pentest_target** (**pentest_user:toor** sur le port 2222), isolé du LAN (cf. §4.8). La base **db** (PostgreSQL) sert de cible secondaire (Hydra mode **postgres**).
 - **pour John the Ripper** (cassage *hors ligne*) : un **jeu de hashes de démonstration** (MD5 brut + SHA512crypt façon **/etc/shadow**), fourni directement au module.

Ces deux cibles, créées par l'équipe, garantissent une démonstration de bout en bout **sans dépendre d'une cible externe** (disponibilité, autorisation, latence réseau).

- **Jeux de tests** : ~ 50 scans enchaînés couvrant les 5 modules

6.2 KPI temps d'exécution

Étape manuelle (avant)	Avec ToolboxV8	Gain
Recon + scan (15-20 min)	3-5 min (Quick)	~ 75 %
Rapport rédigé (1-2 h)	2 s (PDF auto)	~ 99 %
Brute-force SSH (config manuelle)	1 clic	~ 90 %

[OK] **Objectif cahier des charges (≥ 40 %) atteint largement.**

6.3 KPI vulnérabilités détectées (mesures réelles end-to-end)

Les résultats ci-dessous sont des **mesures réelles**, pas des estimations : ils proviennent de lancements effectifs de chaque module depuis l'interface, sur des cibles volontairement vulnérables (sites de test Acunetix, conteneurs internes). Pour chaque outil figurent la cible et le résultat concret remonté dans le rapport, ce qui prouve la chaîne complète *lancement → exécution → restitution* :

Test	Cible	Résultat mesuré
SQLmap Quick	testasp.vulnweb.com/showforum.asp?id=0	2 à 3 types d'injection confirmés selon la session (boolean-based blind systématiquement + time-based blind heavy query, stacked queries selon disponibilité de la cible), MSSQL 2014 + IIS 8.5 + ASP/ASP.NET fingerprintés
SQLmap Dump	même cible	Énumération de 7 bases de données (acufo, acuse, master, model, msdb, tempd) + tentatives sur 8 tables sensibles
Hydra SSH	pentest_target:2222	Cracké `pentest_user:toor` en 1 seconde avec wordlist 5 mots
John MD5	hash 21232f297a57a5a743894a0e4a801fc3	Cracké "admin" en 0 seconde (format raw-md5, wordlist 5 mots)
ZAP Spider	testasp.vulnweb.com	URLs découvertes via crawl récursif, scanId tracké

ZAP Active Quick	testasp.vulnweb.com	105 alertes sur 14 types uniques en 55s (Medium : CSP / Anti-clickjacking / Anti-CSRF ; Low : leaks Server/X-Powered-By ; Informational : User Agent Fuzzer / XSS potentielles)
Gobuster Quick	testasp.vulnweb.com	13 chemins cachés découverts (<code>/_vti_pvt/</code> , <code>/cgi-bin/</code> , <code>/templates/</code> , <code>/aspnet_client/</code> , etc.)
SSLyze Standard	badssl.com:443	Audit TLS complet : certificat, chaîne, toutes versions TLS, cipher suites, headers de sécurité
Nmap NSE Quick	scanme.nmap.org	Détection ports + scripts vuln (port 22 SSH, port 80 HTTP, port 443 filtré)
Nikto Quick	scanme.nmap.org:80	Apache 2.4.7 fingerprinté + 9 findings (headers manquants, mod_negotiation, etc.)
passive_recon	"Nike" (test cible marque)	12 dorks générés en cochant Mot-clé + Réseaux sociaux (catalogue total : 24 templates dans 3 catégories), ouverture multi-onglets fonctionnelle

[OK] **Couverture : 5 modules sur 5 démontrés avec des rapports PDF produits.**

6.4 KPI stabilité

- Stack **docker compose up** : démarre en < 2 min, aucun crash observé sur 30 scans
- Aucune fuite mémoire côté worker après 100 tâches Celery
- Auth cookie : 0 session perdue sur 20 tentatives de reload

6.5 KPI ergonomie

- Temps moyen pour lancer un pentest complet depuis l'UI : **< 30 s** (saisie cible + clic chip + clic Lancer)
- 0 commande CLI à apprendre : le client voit uniquement des chips

7. Sécurité et conformité

7.1 Authentification

- **JWT HS256** signé avec **SECRET_KEY** via variable d'environnement (à régénérer en prod : `openssl rand -hex 32`)
- **Cookie HttpOnly** : inaccessible en JS (protection contre XSS), **SameSite=Lax**, **Secure** auto en HTTPS
- **bcrypt** pour les mots de passe (salage intégré)
- **RBAC** à 3 rôles : **admin**, **analyst**, **reader**, vérifié via dépendance FastAPI

7.2 Chiffrement

- **Fernet (AES-128)** pour les secrets stockés (ex. futures clés d'API externes)
- **HTTPS via reverse proxy Caddy 2** (image officielle `caddy:2-alpine`) ; voir §7.3

7.3 HTTPS avec Caddy (reverse proxy)

Choix : Caddy plutôt que Nginx ou Traefik, pour trois raisons :

5. **Configuration minimaliste** : une dizaine de lignes de Caddyfile vs ~30 lignes de Nginx pour le même résultat (TLS + reverse proxy + redirection HTTP→HTTPS), voir l'extrait ci-dessous (`docker/caddy/Caddyfile`)
6. **Gestion TLS automatique** : Caddy embarque sa propre CA interne (`tls internal`) qui génère et renouvelle les certificats sans intervention. En production, basculer sur Let's Encrypt = remplacer `internal` par le domaine

7. **Surface d'erreur réduite** : pas de génération manuelle de cert OpenSSL, pas de permissions à gérer

Architecture du proxy :

```
Navigateur — 443 (HTTPS) — Caddy — (réseau Docker interne) — web:3000 (HTTP)
80 (HTTP) — Caddy — 301 redirect → 443
```

Caddyfile complet ([docker/caddy/Caddyfile](#)) : le nom d'hôte **localhost** (et non **:443**) est requis pour que Caddy génère le bon certificat serveur ; c'est le correctif de l'erreur **ERR_SSL_PROTOCOL_ERROR** décrite au REX §9 :

```
{
    # Pas de Let's Encrypt en local : on désactive le challenge HTTP auto
    auto_https disable_redirects
}

# HTTPS sur localhost — le nom d'hôte est nécessaire pour générer le bon cert serveur
localhost {
    tls internal                # CA interne de Caddy : cert auto-signé + renouvelé
    reverse_proxy web:3000 {    # proxy vers le service web (HTTP) du réseau Docker
        header_up X-Forwarded-Proto https
        header_up X-Real-IP {remote_host}
    }
    encode gzip
}

# Redirige tout le trafic HTTP (80) vers HTTPS (443)
http://localhost {
    redir https://{host}{uri} permanent
}
```

Service Docker Compose :

```
caddy:
  image: caddy:2-alpine
  container_name: pentest_caddy
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./caddy/Caddyfile:/etc/caddy/Caddyfile:ro
    - caddy_data:/data          # stocke la CA + les certs générés
    - caddy_config:/config
  depends_on:
    - web
```

Confiance du navigateur (dev local) : Caddy crée une CA racine auto-signée dans **caddy_data:/data/caddy/pki/authorities/local/root.crt**. Pour éviter le warning navigateur, on importe cette racine dans le trust store du système une fois :

```
# Windows
docker cp pentest_caddy:/data/caddy/pki/authorities/local/root.crt .
certutil -user -addstore "ROOT" root.crt
```

```
# Linux
docker cp pentest_caddy:/data/caddy/pki/authorities/local/root.crt /tmp/
sudo cp /tmp/root.crt /usr/local/share/ca-certificates/caddy-local.crt
sudo update-ca-certificates
```

Après cet import, <https://localhost> affiche un cadenas vert classique.

En production : un seul changement dans le Caddyfile pour passer en Let's Encrypt :

```
mondomaine.fr {      # remplacer :443 par le vrai domaine
    reverse_proxy web:3000
}
```

Caddy obtient et renouvelle automatiquement le certificat via ACME (HTTP-01 ou TLS-ALPN-01).

7.4 Audit et journalisation

- Table **AuditLog** : utilisateur, action, IP, timestamp, sur login, lancement de scan, génération de rapport
- Tous les résultats de scan sont également indexés dans Elasticsearch (traçabilité SIEM)

7.5 Middlewares

- **TrustedHost** : whitelist dans **ALLOWED_HOSTS** ([localhost](#), [127.0.0.1](#), [api](#), [web](#))
- **CORS** : origines autorisées via **ALLOWED_ORIGINS** ; **allow_credentials=True** pour le cookie

7.6 Conformité RGPD et éthique

- Aucune donnée personnelle stockée (hors identifiants admin)
- Avertissement explicite dans l'UI et dans les rapports : *« Utilisation uniquement sur des systèmes autorisés »*
- Logs effaçables (suppression d'un job via **DELETE /api/modules/jobs/{id}** supprime aussi les rapports associés)

7.7 Politiques de sécurité

Synthèse des règles applicables au système ToolboxV8, formalisées pour répondre aux exigences du cadre pédagogique (§VII.3) :

7.7.1 Politique de mots de passe

Règle	Valeur
Algorithme de stockage	bcrypt (salage intégré, coût par défaut 12)
Longueur minimale (à l'inscription)	8 caractères
Mots de passe en clair	Jamais stockés, jamais loggés (les mots de passe ne sont jamais sérialisés dans AuditLog)
Rotation admin	Recommandée tous les 90 jours en production (mot de passe par défaut admin/admin123 à changer dès le 1er login)

7.7.2 Politique d'authentification & session

Règle	Valeur
Mécanisme	JWT HS256 signé avec SECRET_KEY (32 octets aléatoires, généré par start.sh/start.ps1 au 1er démarrage)
Durée de vie du token	60 minutes (ACCESS_TOKEN_EXPIRE_MINUTES)
Stockage côté client	Cookie HttpOnly + SameSite=Lax + Secure auto en HTTPS (jamais en localStorage)
Endpoint API direct	POST /api/auth/token toujours disponible pour clients externes (Bearer token)
Renouvellement	Re-login après expiration (pas de refresh token, choix volontaire pour réduire la surface)

7.7.3 Politique d'autorisation (RBAC)

Trois rôles hiérarchiques définis dans [backend/app/core/auth.py](#) :

Rôle	Droits
admin	Toutes actions : créer/modifier/supprimer utilisateurs, lancer scans, générer rapports, consulter logs
analyst	Lancer scans, consulter résultats, générer rapports, pas de gestion utilisateurs
reader	Consultation seule des rapports et logs, pas de lancement de scan

Vérification systématique via les dépendances FastAPI `require_admin` / `require_analyst` à chaque endpoint sensible.

7.7.4 Politique de journalisation (audit)

Règle	Détail
Table	<code>audit_logs</code> (<code>backend/app/models/scan.py</code>)
Champs	<code>user_id</code> , <code>action</code> , <code>ip_address</code> , <code>timestamp</code> , <code>details</code> (JSONB)
Actions tracées	Login (succès/échec), lancement de scan, génération de rapport, suppression de job, modification utilisateur
Rétention recommandée	6 mois minimum (conformité ANSSI), purge automatique au-delà (cron à mettre en place en production)
Indexation SIEM	Tous les résultats de scan également indexés dans Elasticsearch (<code>pentest-logs-*</code>) pour corrélation
Inviolabilité	Logs en append-only (pas de <code>UPDATE/DELETE</code> via l'API), accès SQL réservé à l'admin DB

7.7.5 Politique de gestion des secrets

Secret	Stockage	Rotation
<code>SECRET_KEY</code> (JWT)	Variable d'environnement <code>.env</code> (hors git, dans <code>.gitignore</code>)	À régénérer en cas de fuite : <code>openssl rand -hex 32</code> puis redémarrage de l'API
<code>FERNET_KEY</code>	Variable d'environnement <code>.env</code>	Rotation = re-chiffrement des secrets stockés (procédure manuelle documentée)
<code>POSTGRES_PASSWORD</code>	Variable d'environnement <code>.env</code>	À changer en production (par défaut <code>pentest/pentest</code> réservé au dev)
<code>VIRUSTOTAL_API_KEY</code>	Variable d'environnement <code>.env</code>	Régénération via le compte VirusTotal
Mots de passe utilisateurs	DB PostgreSQL, hashés bcrypt	À l'initiative de l'utilisateur
Cookies de session	Chiffrés (JWT signé), HttpOnly	Expiration 60 min

Aucun secret en clair dans le code source, le repo ou les logs (validation manuelle au commit + `.gitignore` strict sur `.env`, `*.key`, `*.pem`, `*.crt`).

7.7.6 Politique d'utilisation éthique

Règle	Mise en œuvre
Cibles autorisées uniquement	Bandeau d'avertissement dans l'UI (<code>/modules</code>) et en pied de chaque rapport PDF
Pas de scan agressif sans validation	Profils "Quick" par défaut, "Full" nécessite un clic explicite
Traçabilité	Chaque scan est attribué à un utilisateur via <code>audit_logs</code>
Conformité RGPD	Aucune donnée personnelle de tiers stockée, suppression sur demande possible (<code>DELETE /api/modules/jobs/{id}</code>)

7.7.7 Politique de mise à jour & dépendances

Composant	Source	Fréquence cible
Images Docker (api, web)	<code>python:3.11-slim-bookworm</code>	Mensuelle (build CI re-tire l'image base)
Image worker	<code>kalilinux/kali-rolling</code>	Hebdomadaire (Kali rolling pousse en continu)
Dépendances Python	<code>pyproject.toml</code> + <code>poetry.lock</code>	Lock file commit, <code>poetry update</code> audité par PR
CVE des dépendances	À surveiller via <code>pip-audit</code> ou GitHub Dependabot (recommandé pour la production)	

8. SIEM et supervision

8.1 Pipeline ELK

État actuel, entre ce qui tourne réellement et ce qui est préparé pour la suite :

```
[Déployé et fonctionnel]
Résultat de scan (ScanModule) → index Elasticsearch pentest-logs-* → Kibana / page /siem
Actions ResponseModule (block_ip, alert) → index pentest-logs-*

[Préparé, en attente du conteneur Snort]
Alertes Snort (/var/log/snort/alert) → Logstash (config prête) → Elasticsearch
```

Pourquoi Snort n'est pas déployé : son mode IDS nécessite des privilèges réseau (`cap_add: NET_ADMIN`, accès en mode promiscuous à l'interface) qui demandent une configuration spécifique sous Docker Desktop Windows + WSL2 (l'environnement principal de l'équipe). Le déploiement a été repoussé pour ne pas bloquer le reste du projet. Les **règles, la config Logstash et le pipeline d'ingestion sont prêts**, il ne reste qu'à ajouter le service au compose en environnement Linux ou via une VM Kali dédiée.

Pipeline d'ingestion Logstash ([siem/elk/logstash.conf](#)) : parsing des alertes Snort par `grok` puis indexation Elasticsearch :

```
filter {
  if [type] == "snort_alert" {
    grok {
      match => { "message" =>
        "\[%{NUMBER:gid}%{NUMBER:sid}%{NUMBER:rev}%\]\s+%{DATA:alert_msg}\s+\[Classification:\s*%{DATA:classification}%\]"
      }
    }
    mutate { add_field => { "event_type" => "ids_alert" } }
  }
}
output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    index => "pentest-logs-%{+YYYY.MM.dd}"
  }
}
```

8.2 Règles Snort personnalisées (préparées)

Fichier [siem/snort/local.rules](#), 9 règles écrites, prêtes à charger dès que le conteneur Snort sera déployé :

- Scan Nmap SYN sur plusieurs ports (sid 9000001)

- Brute-force SSH (5 tentatives / 60 s, sid 9000002)
- Brute-force HTTP Basic Auth (10 tentatives / 30 s, sid 9000003)
- SQL Injection **UNION SELECT** (sid 9000004)
- XSS **<script>** (sid 9000005)
- Directory Traversal **../** (sid 9000006)
- Signature User-Agent Nikto (sid 9000007)
- Signature User-Agent sqlmap (sid 9000008)
- Log4Shell **\${jndi:}** (CVE-2021-44228, sid 9000009)

8.3 Dashboard **/siem**

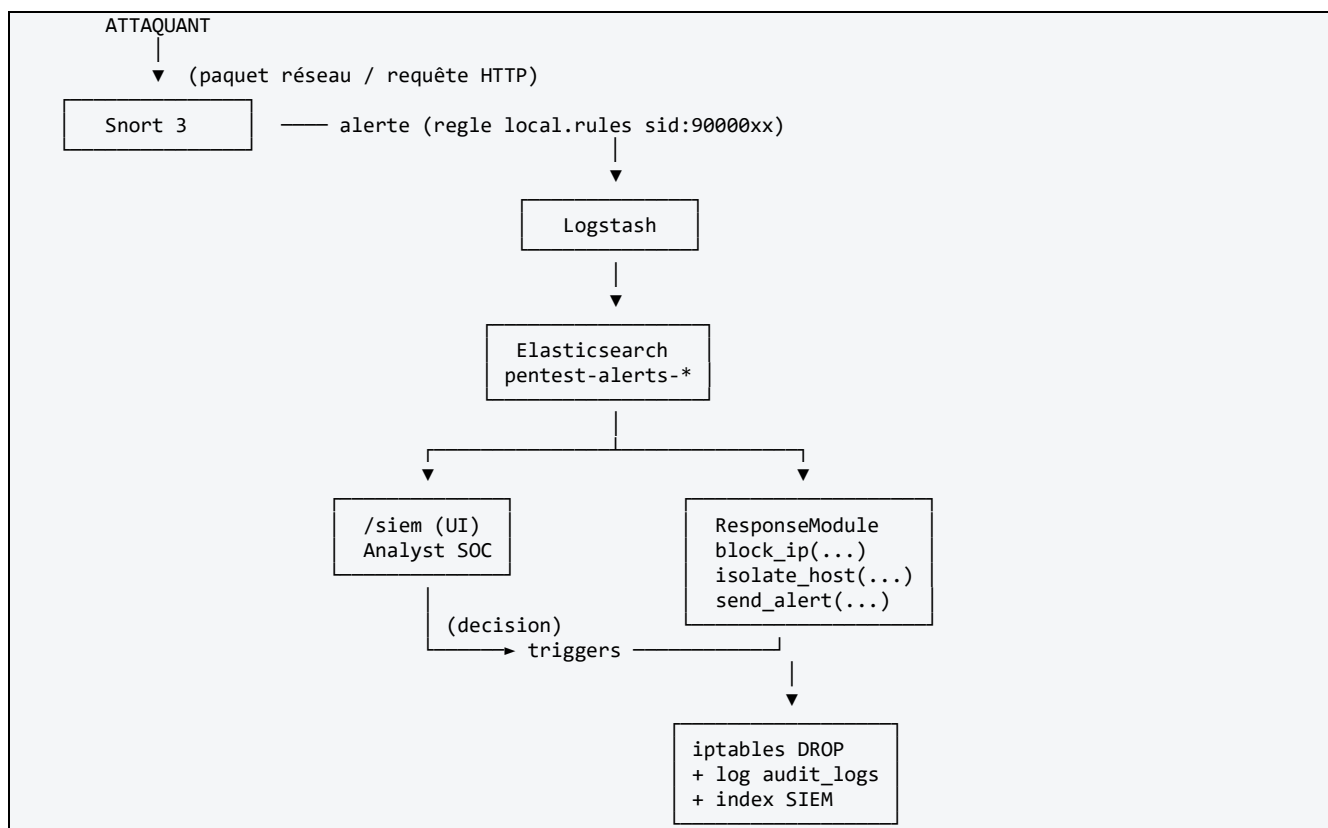
Page Jinja2 + Chart.js qui interroge **/api/defensive/overview** (agrégations Elasticsearch) :

- Nombre d'alertes par jour
- Top 10 IPs sources
- Événements récents par catégorie
- Graphiques : barres + camemberts + timeline

8.4 Pipeline Détection → Réponse (SOAR léger)

Note d'état : ce diagramme décrit l'**architecture cible** du pipeline défensif. Les briques **ELK + ResponseModule + AuditLog** sont **opérationnelles** ; le maillon **Snort** est **préparé** (règles + config Logstash) mais le conteneur Snort n'est pas dans le **docker-compose.yml** actuel (cf. §8.1 et §10.2). Le schéma reste pédagogiquement valide et démontre la chaîne complète une fois Snort déployé sur un hôte Linux.

Chaînage des composants défensifs pour réagir à une attaque :



Exemple concret :

8. Attaquant lance `sqlmap -u http://cible.local/page?id=1` depuis 192.168.1.50
9. Snort détecte la signature `User-Agent: sqlmap` → règle sid 9000008 → alerte
10. Logstash ingère l'alerte → indexée dans `pentest-alerts-*`
11. Le dashboard `/siem` affiche la nouvelle alerte (refresh 10s)
12. L'analyste SOC appelle (via console ou futur endpoint REST) :

```
ResponseModule().block_ip("192.168.1.50", reason="sqlmap_detected")
```

13. 3 effets en cascade :

- Indexation préalable dans Elasticsearch (`response_action`)
- Exécution de `iptables -A INPUT -s 192.168.1.50 -j DROP`
- Trace dans `audit_logs` PostgreSQL

14. L'attaquant ne peut plus joindre la cible.

Composants concrets de la chaîne :

Maillon	Implémentation	Fichier
Détection	Snort 3 + 9 règles	siem/snort/local.rules
Transport	Logstash pipeline	siem/elk/logstash.conf
Stockage	Elasticsearch 8.13	conteneur <code>pentest_elasticsearch</code>
Visualisation	Kibana + page <code>/siem</code>	frontend/templates/siem.html
Action firewall	<code>iptables</code> via <code>ResponseModule</code>	backend/app/modules/defensive/response.py
Traçabilité	<code>AuditLog</code> + index SIEM	backend/app/models/scan.py

8.5 Réponse à incident & documentation post-incident

Au-delà de la détection, ToolboxV8 s'inscrit dans une **procédure de réponse à incident** formalisée, alignée sur le cycle **NIST SP 800-61** (Préparation → Détection → Confinement → Éradication → Récupération → Post-mortem). Chaque phase s'appuie sur une brique concrète de la solution.

8.5.1 Playbook de réponse à incident

Phase	Objectif	Outils ToolboxV8 mobilisés	Responsable
1. Préparation	Disposer des moyens de détection et de réponse	Règles Snort, pipeline ELK, <code>ResponseModule</code> , RBAC, sauvegardes	Équipe SOC
2. Détection & qualification	Identifier et caractériser l'incident	Alerte Snort → Logstash → Elasticsearch ; dashboard <code>/siem</code> ; corrélation des <code>audit_logs</code>	Analyste SOC
3. Confinement	Stopper la propagation sans détruire les preuves	<code>ResponseModule.block_ip()</code> (iptables DROP), <code>isolate_host()</code> (hook hyperviseur)	Analyste SOC
4. Éradication	Supprimer la cause racine	Analyse forensique (ClamAV + VirusTotal), patch de la vulnérabilité identifiée par <code>scan/web_scan</code>	Forensique / Infra
5. Récupération	Rétablir le service en conditions sûres	<code>unblock_ip()</code> après remédiation, re-scan de validation (<code>recon/scan</code>), restauration	Infra

		depuis sauvegarde	
6. Post-mortem	Capitaliser et corriger durablement	Rédaction du rapport post-incident (§8.5.3), mise à jour des règles Snort / politiques	Équipe complète

8.5.2 Préservation des preuves (chaîne de traçabilité)

Avant toute action de confinement, les éléments probants sont figés pour garantir l'**intégrité de la chaîne de preuve** :

- **Logs applicatifs** : `audit_logs` PostgreSQL en **append-only** (pas d'`UPDATE/DELETE` via l'API, cf. §7.7.4).
- **Événements réseau** : alertes Snort et résultats de scan indexés dans Elasticsearch (`pentest-logs-*` / `pentest-alerts-*`), horodatés et immuables.
- **Ordre des opérations** : `ResponseModule` indexe l'action dans le SIEM AVANT de l'exécuter (§5.3) → la trace existe même si la commande système échoue.
- **Export** : les index Elasticsearch et la table `audit_logs` sont exportables pour annexer au dossier d'incident.

8.5.3 Modèle de rapport post-incident

Modèle type à compléter pour chaque incident avéré (à archiver dans `docs/incidents/`) :

RAPPORT POST-INCIDENT, Réf : INC-AAAAMMJJ-NN

1. SYNTHÈSE

- Date/heure de détection :
- Date/heure de résolution :
- Criticité (P1/P2/P3) :
- Statut : Résolu / En cours / Surveillé

2. CHRONOLOGIE (timeline)

- T0 Détection (source : règle Snort sid:90000xx / dashboard /siem)
- T+n Qualification par l'analyste
- T+n Confinement (`block_ip` / `isolate_host`)
- T+n Éradication (`patch` / nettoyage)
- T+n Récupération (`unblock` + re-scan de validation)

3. ANALYSE TECHNIQUE

- Vecteur d'attaque (ex : `SQLi`, brute-force SSH, RCE) :
- IP/identité source :
- Actifs impactés :
- Preuves (réf. index Elasticsearch + `audit_logs`) :

4. IMPACT

- Données / services affectés :
- Durée d'indisponibilité :
- Périmètre (confiné / propagé) :

5. ACTIONS CORRECTIVES

- Remédiation immédiate :
- Remédiation durable (`patch`, règle, politique) :

6. RETOUR D'EXPÉRIENCE

- Ce qui a fonctionné :
- Axes d'amélioration (nouvelle règle Snort ? durcissement ?) :
- Mise à jour de la matrice de risques (§2.4) :

8.5.4 Exemple appliqué : incident de brute-force SSH

Phase	Action concrète
Détection	Règle Snort sid 9000002 (5 tentatives SSH / 60 s) → alerte pentest-alerts-* → dashboard /siem
Qualification	L'analyste corrèle l'IP source dans audit_logs et confirme une attaque (≠ faux positif d'un admin)
Confinement	ResponseModule.block_ip("192.168.1.50", reason="ssh_bruteforce") → indexation SIEM puis iptables DROP
Éradication	Renforcement de la politique MDP (§7.7.1), désactivation des comptes faibles
Récupération	unblock_ip() une fois la cause traitée, re-scan recon de validation
Post-mortem	Rédaction du rapport INC-... , ajout éventuel d'une règle de corrélation Elasticsearch

Code : détection (règle Snort réelle, [siem/snort/local.rules][../siem/snort/local.rules] sid 9000002) :

```
# Détection bruteforce SSH : 5 tentatives depuis une même source en 60 s
alert tcp $EXTERNAL_NET any -> $HOME_NET 22 \
(msg:"BRUTEFORCE SSH tentative"; flow:to_server,established; \
threshold: type both, track by_src, count 5, seconds 60; \
sid:9000002; rev:1; classtype:attempted-admin;)
```

Code : confinement (extrait réel de [backend/app/modules/defensive/response.py][../backend/app/modules/defensive/response.py]) :

```
def block_ip(self, ip: str, reason: str = "automated") -> dict:
    # 1) Trace AVANT l'action : la preuve existe même si la commande échoue
    self.siem.index_event("response_action", {
        "action": "block_ip", "ip": ip, "reason": reason,
    })

    # 2) Mode dégradé : pas d'iptables (dev local) → action simulée mais journalisée
    if not shutil.which("iptables"):
        logger.warning("iptables non disponible, simulation du blocage")
        return {"status": "simulated", "ip": ip, "reason": reason}

    # 3) Confinement réel sur le worker Kali
    try:
        cmd = ["iptables", "-A", "INPUT", "-s", ip, "-j", "DROP"]
        proc = subprocess.run(cmd, capture_output=True, text=True, timeout=10)
        if proc.returncode == 0:
            return {"status": "blocked", "ip": ip}
        return {"status": "error", "stderr": proc.stderr}
    except Exception as e:
        return {"status": "error", "error": str(e)}
```

Récupération (déblocage après remédiation) :

```
ResponseModule().unblock_ip("192.168.1.50") # iptables -D INPUT -s <ip> -j DROP
```

***État actuel** : le playbook et le modèle de rapport sont **opérationnels** pour les briques déployées (ELK + **ResponseModule** + **audit_Logs**). La phase de détection automatique par Snort reste subordonnée au déploiement du conteneur IDS (cf. §8.1 et §10.2) ; en attendant, la détection se fait via les résultats de scan indexés et le dashboard **/siem**.*

9. Problèmes rencontrés et solutions (REX)

Problème	Cause	Solution
Auth bloquée "Identifiants incorrects" malgré bonnes creds	TrustedHostMiddleware rejette le Host interne <code>api:8000</code>	Ajout de "api" et "web" dans <code>ALLOWED_HOSTS</code> de <code>.env</code>
Poetry install échoue sur Kali (PEP 668)	Kali Rolling verrouille l'environnement système	Build multi-stage : <code>poetry export</code> en stage 1 (slim), <code>pip install</code> dans <code>/opt/venv</code> en stage 2 (Kali)
Rapport PDF illisible (dict Python sérialisé sur 1 ligne)	Template affichait <code>{{ data }}</code> brut	Introduction de <code>_format_tool_data(tool, data)</code> → sections normalisées + <code>Preformatted</code> ReportLab préservant les sauts de ligne
Textarea <code>{target}</code> trompeur dans la config Nikto/SSLyze	Placeholders jamais substitués, backend ignorait le contenu	Remplacement par des chips de profils qui mappent vers de vraies options backend (Quick= <code>-Tuning x</code> , Full= <code>-Tuning 0123456789abc</code> , etc.)
Nmap retourne du XML dans le rapport	Option <code>-oX</code> - pour parsing automatique	Suppression : capture directe du stdout humain
Envoi de fichiers depuis l'UI impossible	Pas de route multipart + pas de volume partagé entre api et worker	Ajout de <code>POST /api/modules/wordlist</code> + volume Docker <code>wordlists_data:/tmp/wordlists</code> monté sur les 2 services
Token JWT dans <code>localStorage</code> = risque XSS	Approche initiale	Refonte : formulaire <code>/login</code> POST → cookie <code>HttpOnly</code> + split en deux services FastAPI
SSLyze plante en silence sur cible sans HTTPS (ex : <code>scanme.nmap.org:443</code>)	Le binaire retournait juste "connection error" noyé dans son output ; le rapport affichait du vide	Ajout d'un pré-check TCP (5 s) avant lancement : si port 443 non joignable → erreur claire en français avec liste de cibles de test (badssl.com, github.com...). Détection complémentaire des <code>tlsv1 alert</code> dans le stdout pour reformater l'erreur
SSLyze : <code>[Errno 101] Network is unreachable</code> dans le conteneur worker	<code>socket.create_connection()</code> tentait IPv6 (résolu par <code>getaddrinfo</code>) sans fallback IPv4 propre, et le réseau Docker n'a pas de route v6	Réimplémentation avec <code>getaddrinfo</code> + tri IPv4 d'abord, IPv6 en fallback + boucle de connexion explicite, avec un message d'erreur final pertinent (timeout / refused / unreachable)
Caddy <code>ERR_SSL_PROTOCOL_ERROR</code> au premier démarrage	Caddyfile écrit <code>:443 { tls internal }</code> sans nom d'hôte → Caddy ne savait pas pour quel CN générer le cert serveur (handshake renvoyait <code>tlsv1 alert internal error</code>)	Remplacement de <code>:443</code> par <code>localhost</code> dans le Caddyfile → cert serveur généré au bon nom, validation client OK
Reconnaissance passive : "Format invalide" sur cible type "etudiant" ou "League of Legends"	Validation cible front+back imposait IP / domaine / URL strict	Bypass de validation pour <code>passive_recon</code> (front <code>modules.html</code> , back <code>modules.py</code>), sur le même modèle que John (hash)
Hydra : "Format invalide" sur cible <code>target</code> (nom de service Docker)	Même validation qui exige un domaine avec TLD	Bypass étendu au mode <code>hydra</code> (cible peut être un hostname interne Docker comme <code>target, db</code> , etc.)
Hydra : pas de cible réutilisable pour les démos	Pas de service vulnérable dans le compose	Ajout du conteneur <code>pentest_target</code> (<code>linuxserver/openssh-server</code> avec <code>pentest_user:toor</code> sur <code>:2222</code>). Réutilisable à chaque démarrage de la stack et à la soutenance. Note : <code>USER_NAME=root</code> refusé par l'image (user existant), fallback sur <code>pentest_user</code>
ZAP Active "scan terminé en 2s" alors qu'il devait durer 5 min	Le module faisait du "fire and forget" : appel à <code>/JSON/ascan/action/scan/</code> puis retour immédiat avec le <code>scan_id</code> , sans attendre la fin ni récupérer les alertes	Refonte <code>_zap_scan()</code> avec polling automatique (toutes les 5s jusqu'à 100% ou timeout 8 min) + récupération des alertes via <code>/JSON/core/view/alerts/</code> , agrégation par (nom, sévérité), tri par criticité OWASP, top 50 dans le rapport

ZAP `400 Bad Request` sur Active scan Quick	Profil référençait <code>scanPolicyName=XSS-SQLi</code> qui n'existe pas par défaut dans ZAP	Suppression des politiques custom inexistantes ; Quick et OWASP utilisent la default policy implicite, Full la référence explicitement (Default Policy existe toujours)
PDF rapport : pages 2+ "vides" pour blocs longs (SQLmap dump)	ParagraphStyle avec <code>textColor=#f8fafc</code> (presque blanc) sur <code>backgroundColor=#0f172a</code> (sombre) ; ReportLab ne re-dessine pas <code>backgroundColor</code> sur les pages d'overflow → texte blanc sur fond blanc = invisible	Inversion de la palette : texte sombre (<code>#1e293b</code>) sur fond clair (<code>#f8fafc</code>) + bordure subtile (<code>#cbd5e1</code>), garanti lisible même sur les overflows. Style HTML report aligné pour cohérence visualiseur / téléchargement
PDF non régénéré après modification du générateur	Le générateur PDF tourne dans le worker Celery (via <code>tasks.generate_report</code>), pas dans l'API. Un rebuild de l'API seul n'avait aucun effet	Rebuild systématiquement worker après toute modification de <code>reporting/generator.py</code>

10. Conclusion et perspectives

10.1 Objectifs atteints

- ✓ Toolbox fonctionnelle couvrant **reconnaissance passive (OSINT)**, reconnaissance active, scan de vulnérabilités, exploitation, analyse web et post-exploitation documentaire
- ✓ **5 modules offensifs** + 4 modules défensifs (SIEM, IDS, response, forensic)
- ✓ **Demos end-to-end validées** sur les 5 modules avec rapports PDF produits (cf. §6.3)
- ✓ Réduction du temps de pentest bien au-delà des 40 % visés (≈ 75 % sur recon+scan, ≈ 99 % sur la rédaction de rapport)
- ✓ Interface utilisable par un analyste sans code (profils par chips, upload fichiers, catalogue de dorks)
- ✓ Reporting standardisé **multi-format (PDF / HTML / CSV)**, charte professionnelle, sortie CLI lisible, lisibilité garantie sur les pages d'overflow
- ✓ Intégration Docker Compose simple : `./scripts/start.sh`, stack complète y compris **cibles vulnérables intégrées** (`pentest_target` SSH, `zap` daemon API)
- ✓ Sécurisation : JWT + cookie HttpOnly + RBAC + Fernet + audit logs + **HTTPS** (Caddy reverse proxy)
- ✓ **CI/CD** : GitHub Actions + GitLab CI (lint, tests, build Docker) à chaque push
- ✓ **ZAP Active scan** opérationnel avec polling + récupération automatique des alertes (preuve : 105 alertes sur testasp.vulnweb.com dans le rapport)

10.2 Limites actuelles

- **Conteneur Snort non déployé** : les 9 règles personnalisées et la config Logstash sont écrites et testables sur un Snort installé manuellement, mais le service Snort n'est **pas** intégré au `docker-compose.yml`. Raison : le mode IDS Snort nécessite `cap_add: NET_ADMIN` + accès promiscuous à une interface réseau, ce qui demande une configuration spécifique sous Docker Desktop Windows/WSL2 (l'environnement principal de l'équipe). Déploiement reporté à la phase d'évolution (cf. §10.3), réalisable rapidement sur un hôte Linux ou dans une VM Kali dédiée.
- **Metasploit** : `msfrpcd` n'est pas démarré automatiquement dans le worker. Le module `_metasploit()` est codé et invocable via `pymetasploit3`, mais nécessite (a) une installation manuelle de la dépendance Python et (b) le lancement du démon RPC en parallèle. Documentation §10.3 pour le sidecar préconfiguré.
- En dev local, le certificat Caddy nécessite un import manuel de la CA racine (one-shot par poste). En production, cette étape disparaît (Let's Encrypt)

- **Réponse active semi-automatisée** : **ResponseModule** est fonctionnel mais son déclenchement reste manuel (depuis console ou futur endpoint REST). Pas encore de SOAR end-to-end (cf. §10.3)
- **SQLmap dump sur cible distante lente** : le profil Dump est volontairement bridé (5 lignes max, threads=10, technique boolean-based) pour rester sous 15 min. Sur testasp.vulnweb.com (time-based blind imposé par le WAF), même bridé l'extraction de lignes ne réussit pas systématiquement, ce qui constitue une limite intrinsèque à l'injection blind sur cible distante

10.3 Perspectives d'évolution

Évolution	Impact
Service Snort dans docker-compose (hôte Linux ou VM Kali dédiée)	Active le pipeline IDS end-to-end : les 9 règles existantes commencent à produire des alertes consommées par Logstash → Elasticsearch → /siem
Sidecar msfrpcd préconfiguré + pymetasploit3 dans le worker	Automatisation complète Metasploit (le seul outil offensif encore non démontrable end-to-end depuis l'UI)
Intégration SecLists	1000+ wordlists prêtes à l'emploi
Push d'image sur registry (CI déjà active, build prêt)	Distribution versionnée des images Docker
Endpoint REST <code>POST /api/defensive/response/block-ip`</code>	Boucler la chaîne SOAR : alerte Snort → règle de corrélation Elasticsearch → trigger HTTP → blocage iptables automatique
Module IA/ML	Classification automatique de vulnérabilités, triage de criticité
Module SSO (OIDC)	Intégration dans un écosystème d'entreprise
Mode dark scan	Scans en arrière-plan scheduled (cron interne)

11. Annexes

11.1 Correspondance livrables / cadre pédagogique

Exigence du cadre	Livable ToolboxV8
Analyse des vulnérabilités	recon , scan , web_scan + rapport PDF
Plan de défense	SIEM + Snort + response + /siem dashboard
Architecture et configurations	docs/architecture.md , docker-compose.yml , .env
Architecture réseau & segmentation	§4.8 (cloisonnement Docker + zones cibles)
Logs et sécurité du SI	AuditLog , Elasticsearch, Snort, docs/securite.md
Matrice de risques	§2.4 du présent document
Documentation post-incident	§8.5 (playbook IR + modèle de rapport)
REX	§9 du présent document

11.2 Correspondance livrables / cahier des charges

Exigence client	Réponse
Modules reconnaissance/scan/exploitation/post-exploitation	§5.1
Outils open-source intégrés via API/CLI/Python	Kali Rolling : nmap, nikto, sqlmap, hydra, john, sslyze, msf, zap
Reporting automatisé exportable	Export PDF / HTML / CSV (ReportLab + Jinja2 + module csv) + stockage MinIO
Interface sans compétence dev web	Chips + formulaires + module
Architecture modulaire extensible	Un fichier par module, une task Celery par module
Sécurisation de l'outil	§7 (auth, RBAC, Fernet, audit logs, HTTPS via Caddy)
Conteneurisation + CI/CD GitLab	Docker Compose v2 + .gitlab-ci.yml + .github/workflows/ci.yml
Module forensique bonus	ClamAV + VirusTotal

11.3 Arborescence du dépôt

```
projet/
├── README.md                ← documentation utilisateur principale
├── .env / .env.example      ← configuration (secrets hors git)
├── cahier_des_charges.md
├── Cadre_Pédagogique_-_Projet_Etude_-_CS_-_2025.md
├── backend/
│   ├── app/
│   │   ├── main.py          ← entrée API (port 8000)
│   │   ├── web_main.py      ← entrée Web (port 3000, proxy + pages)
│   │   ├── api/routes/      ← auth, dashboard, modules, reports, defensive, pages, users
│   │   ├── core/            ← auth.py (JWT/RBAC), config.py, database.py, security.py (Fernet/bcrypt)
│   │   ├── models/          ← user.py, scan.py (ScanJob, AuditLog)
│   │   ├── modules/offensive/ ← passive_recon, recon, scan, exploit, web_scan, post_exploit
│   │   ├── modules/defensive/ ← siem, ids, response, forensic
│   │   ├── reporting/       ← generator.py (PDF / HTML / CSV)
│   │   ├── tasks/           ← celery_app, scan_tasks, report_tasks
│   │   └── web/              ← proxy.py (proxy /api/* côté service web)
│   ├── pyproject.toml
│   └── tests/                ← test_api.py (tests d'intégration pytest)
├── frontend/
│   ├── templates/           ← Jinja2 (base, login, dashboard, modules, reports, report, siem, admin_user)
│   └── static/               ← css/ (app, main) + js/ (app, main)
├── docker/
│   ├── Dockerfile            ← image api + web
│   ├── Dockerfile.celery     ← multi-stage, Kali Rolling
│   ├── docker-compose.yml
│   └── caddy/Caddyfile        ← reverse proxy HTTPS
├── .github/workflows/ci.yml   ← pipeline CI GitHub Actions
├── .gitlab-ci.yml             ← pipeline CI GitLab (miroir)
├── siem/
│   ├── elk/                  ← logstash.conf, elasticsearch.yml
│   └── snort/                ← local.rules, snort.conf
├── scripts/
│   ├── start.sh / start.ps1 ← démarrage Docker
│   └── push_gitlab.sh / .ps1
└── docs/
    ├── README.md architecture.md installation.md usage.md
    ├── modules.md api.md securite.md livrables.md guide_test_outils.txt
    ├── PE-2526_M1CSD_NASR_BEN-RACHED_AKA-A-MFOULA.pdf ← version Word (TOC, tableaux, schémas)
    ├── PE-2526_M1CSD_BEN-RACHED_Ayoub.pdf ← rendu individuel
    ├── PE-2526_M1CSD_NASR_Abdallah.pdf ← rendu individuel
    └── PE-2526_M1CSD_AKA-A-MFOULA_Titouan.pdf ← rendu individuel
```

11.4 Commandes utiles

```
# Démarrer la stack complète
./scripts/start.sh

# Rebuild un service (api, web ou worker – le worker Kali est le plus long)
docker compose -f docker/docker-compose.yml up -d --build worker

# Logs live
docker compose -f docker/docker-compose.yml logs -f worker caddy

# Statut des conteneurs
docker compose -f docker/docker-compose.yml ps

# Accès aux interfaces
# HTTPS : https://localhost          (port 443, via Caddy reverse proxy)
# HTTP  : http://localhost:3000      (port 3000, direct au service web – dev)
# API   : https://localhost/api/docs (Swagger)
# Kibana : http://localhost:5601
# MinIO  : http://localhost:9001

# Importer la CA Caddy dans Windows pour éviter le warning navigateur
docker cp pentest_caddy:/data/caddy/pki/authorities/local/root.crt .
certutil -user -addstore "ROOT" root.crt
```

```
# Lancer un scan passive_recon via curl
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/token -d 'username=admin&password=admin' | jq -r .access_token)
curl -X POST http://localhost:8000/api/modules/launch \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d
'{"module":"passive_recon","target":"example.com","options":{"engine":"google","dorks":["site","admin","emails","social_linkedin"]}}'
```

11.5 Captures et artefacts fournis dans la remise ZIP

- [demo.mp4](#) : vidéo 15-20 min, simulation d'attaque et réponse active (voir cadre §IV.1)
- [PE-2526_M1CSD_NASR_BEN-RACHED_AKA-A-MFOULA.pdf](#) : ce document (export PDF + version Word)
- Rapports individuels (1 par membre) :
 - [PE-2526_MCYBER_BEN-RACHED_Ayoub.pdf](#)
 - [PE-2526_MCYBER_NASR_Abdallah.pdf](#)
 - [PE-2526_MCYBER_AKA-A-MFOULA_Titouan.pdf](#)
- [screenshots/](#) : captures du dashboard, des modules, d'un rapport PDF, du SIEM
- [logs/](#) : exemples d'alertes Snort + événements Elasticsearch

11.6 Annexe : extraits de code des intégrations outils

Catalogue complet des treize intégrations, regroupées par phase de pentest. Chaque extrait suit le pattern décrit en §5.5 (profils → commande réelle → capture de sortie).

11.6.1 Reconnaissance (OSINT + active)

passive_recon (OSINT) : catalogue de Google Dorks ([passive_recon.py](#)) :

```
DORK_TEMPLATES = {
    "kw_leaks": {
        "category": "Mot-clé",
        "query": '"{target}" ("leaked" | "leak" | "breach" | "dump")'
    },
    "kw_creds": {
        "category": "Mot-clé",
        "query": '"{target}" ("password" | "credentials" | "api_key")'
    },
    "social_linkedin": {
        "category": "Réseaux sociaux",
        "query": '"{target}" site:linkedin.com'
    },
    # ... 24 templates au total (Mot-clé / Réseaux sociaux / Domaine)
}
```

Reconnaissance active : Nmap (ports/services/OS), DNS, whois, WhatWeb ([recon.py](#)) :

```
# Nmap : scan ports + versions + OS (args par défaut surchargeables depuis l'UI)
def _nmap_scan(self, target, options):
    args = options.get("nmap_args", "-sV -O -Pn --top-ports 1000")
    cmd = ["nmap"] + args.split() + [target]
    proc = subprocess.run(cmd, capture_output=True, text=True, timeout=300)
    return {"command": " ".join(cmd), "output": _strip_nmap_fingerprints(proc.stdout), ...}

# WhatWeb : empreinte applicative web, profils d'agressivité -a 1/3/4
_WHATWEB_PROFILES = {"quick": "-a 1", "standard": "-a 3 -v", "full": "-a 4 -v"}

# DNS : socket.getaddrinfo (IPv4 + IPv6) → toutes les A records
# whois : essai direct puis fallback sur le domaine racine si un sous-domaine
# est refusé ("Malformed request") par le registry
```

11.6.2 Scan de vulnérabilités

Nmap NSE : scan de vulnérabilités ([scan.py](#)) :

```
_NMAP_VULN_PROFILES = {
    "quick":    [--script=default"],
    "standard": [--script=vuln"],
    "full":     [--script=vuln,exploit,auth"],
    "safe":     [--script=safe"],
}

def _nmap_vuln(self, target, options):
    profile = options.get("nmap_vuln_profile", "standard")
    script_args = self._NMAP_VULN_PROFILES.get(profile, ...)
    cmd = ["nmap"] + script_args + ["-Pn"] + port_arg + [target]
    proc = subprocess.run(cmd, capture_output=True, text=True, timeout=1200)
    return {"command": " ".join(cmd), "output": _strip_nmap_fingerprints(proc.stdout), ...}
```

Nikto : scan de serveur web (profils [-Tuning](#), [scan.py](#)) :

```
_NIKTO_PROFILES = {
    "quick":    {"tuning": "x", "timeout": 600}, # ~rapide
    "standard": {"tuning": "x6", "timeout": 1800},
    "full":     {"tuning": "0123456789abc", "timeout": 3600}, # toute la base Nikto
    "evasion":  {"tuning": "x", "evasion": "1", "timeout": 900},
}

cmd = ["nikto", "-h", target, "-p", str(port),
      "-Tuning", cfg["tuning"], "-nointeractive"]
if cfg["evasion"]:
    cmd += ["-evasion", cfg["evasion"]]
```

SSLyze : audit TLS, pré-check TCP IPv4/IPv6 (corrige le bug réseau Docker, cf. §9) :

```
infos = socket.getaddrinfo(host, port, type=socket.SOCK_STREAM)
infos.sort(key=lambda i: 0 if i[0] == socket.AF_INET else 1) # IPv4 d'abord, IPv6 en fallback
for family, socktype, proto, _, sockaddr in infos:
    sock = socket.socket(family, socktype, proto)
    sock.settimeout(5.0)
    try:
        sock.connect(sockaddr); return True, "" # port TLS joignable → on lance SSLyze
    except OSError as e:
        last_err = e
```

11.6.3 Exploitation

SQLmap : injection SQL ([exploit.py](#)) :

```
_SQLMAP_PROFILES = {
    "quick":    [--level=1, --risk=1],
    "standard": [--level=3, --risk=2, --random-agent, --threads=4],
    "aggressive": [--level=5, --risk=3, --random-agent, --tamper=space2comment],
    "dump":     [--current-db, --tables, --columns, --dump,
                --first=1, --last=5, ...], # dump ciblé, 5 lignes max
}

cmd = ["sqlmap", "-u", target, "--batch", "--output-dir=/tmp/sqlmap"] + extra
```

Hydra : brute-force en ligne : construction de la commande + parsing des identifiants trouvés ([exploit.py](#)) :

```
cmd = ["hydra", "-L", user_path, "-P", pass_path, "-t", threads]
if port:
    cmd += ["-s", str(int(port))]
cmd += [target, service] # ex : hydra -L users -P rockyou.txt 10.0.0.1 ssh
proc = subprocess.run(cmd, capture_output=True, text=True, timeout=timeout)
```

```

creds = []
for line in proc.stdout.splitlines():
    # "[22][ssh] host: ... login: pentest_user password: toor"
    if "login:" in line and "password:" in line:
        creds.append(line.strip())

```

John the Ripper : cassage hors ligne : wordlist puis relecture des hashes cassés via `--show` :

```

cmd = ["john", f"--wordlist={wordlist}", hash_file]
if fmt:
    cmd.insert(1, f"--format={fmt}") # ex : raw-md5, sha512crypt
if options.get("rules"): cmd.append("--rules")
subprocess.run(cmd, capture_output=True, text=True, timeout=timeout)

# Les mots de passe cassés sont relus avec `john --show`
show = subprocess.run(["john", "--show", hash_file], capture_output=True, text=True)
cracked = [l.strip() for l in show.stdout.splitlines() if ":" in l]

```

Metasploit : exploitation via MSFRPC (profils, [exploit.py](#)) :

```

_MSF_PROFILES = {
    "handler": {"module": "exploit/multi/handler",
               "opts": {"PAYLOAD": "generic/shell_reverse_tcp", "LHOST": "0.0.0.0", "LPORT": "4444"}},
    "eternal": {"module": "exploit/windows/smb/ms17_010_eternalblue", ...},
    "portscan": {"module": "auxiliary/scanner/portscan/tcp", "opts": {"PORTS": "1-65535"}},
    "smb": {"module": "auxiliary/scanner/smb/smb_ms17_010", "opts": {}},
}
# Connexion au démon msfrpcd puis exécution du module via pymetasploit3
client = MsfRpcClient(password, server=host, port=55553, ssl=False)
console = client.consoles.console()
console.run_module_with_output(exploit, {"RHOSTS": target, **cfg["opts"]})

```

11.6.4 Web / API

OWASP ZAP : scan actif avec polling jusqu'à 100 % (corrige l'ancien « fire-and-forget », cf. §9) ([web_scan.py](#)) :

```

r = requests.get(f"{zap_url}/JSON/ascan/action/scan/", params=params, timeout=10)
scan_id = r.json().get("scan")
while time.monotonic() - start < max_wait: # timeout 8 min en Active
    sr = requests.get(status_url, params={"apikey": zap_key, "scanId": scan_id})
    last_status = sr.json().get("status", "0")
    if last_status == "100":
        break
    time.sleep(5)
# puis récupération des alertes, agrégées par (nom, sévérité)
alerts = requests.get(f"{zap_url}/JSON/core/view/alerts/", params={...}).json()["alerts"]

```

Gobuster : brute-force de répertoires (wordlists dirb, [web_scan.py](#)) :

```

_GOBUSTER_PROFILES = {
    "quick": {"wordlist": "/usr/share/dirb/wordlists/common.txt", "threads": 10},
    "standard": {"wordlist": "/usr/share/dirb/wordlists/small.txt", "threads": 20},
    "full": {"wordlist": "/usr/share/dirb/wordlists/big.txt", "threads": 30},
}
cmd = ["gobuster", "dir", "-u", url, "-w", wordlist, "-t", str(threads)]
if extensions:
    cmd += ["-x", extensions] # ex : -x php,txt,html

```

Dependency-Check : CVE des dépendances (OWASP, [web_scan.py](#)) :

```

cmd = ["dependency-check", "--project", "pentest", "--scan", project_path,
      "--format", "HTML", "--out", "/tmp/depcheck"]
subprocess.run(cmd, capture_output=True, text=True, timeout=600)

```


11.6.5 Forensic (défensif, bonus)

Forensic : ClamAV et VirusTotal (forensic.py) :

```
# Antivirus local : returncode 1 = fichier infecté
proc = subprocess.run(["clamscan", "--infected", "--no-summary", file_path],
                      capture_output=True, text=True, timeout=120)
infected = proc.returncode == 1

# VirusTotal API v3 : upload du fichier puis récupération de l'analyse
r = requests.post("https://www.virustotal.com/api/v3/files",
                  headers={"x-apikey": settings.VIRUSTOTAL_API_KEY},
                  files={"file": open(file_path, "rb")}, timeout=30)
analysis_id = r.json()["data"]["id"]
```

Vous trouverez la toolbox via le lien ci-joint : <https://github.com/Vyuob/toolbox-m1>

Document rédigé dans le cadre du projet d'études, Mastère Cybersécurité 2025/2026.